

ContextRuntime:

Cache-Aware Admission and Lifetime Control for Token-Efficient Coding Agents

Ankit Chauhan

InfyU Labs

Abstract

Long-horizon coding agents repeatedly re-send a growing context. Removing history can reduce context workload but may invalidate a discounted prompt cache and create a more expensive rewrite; loading all tool schemas up front can impose the same overhead on every call. We present ContextRuntime, which pairs pre-request admission control for unused tool schemas (in the evaluated configuration, a client launch flag rather than a gateway function) with a gateway that performs forward-only retirement of superseded or cold tool results, bounded retention of prior thinking blocks, persistent mutation, and cache-aligned scheduling that fires at a cold boundary or when expected future cache-read savings exceed the suffix rewrite cost. As shipped, the break-even branch cannot fire on the Anthropic cache profiles. The gateway is off by default, supports observe-before-enforce deployment, and fails open to the original request. All live evidence comes from one repository (Django), one model, and one client version. B6 is a preregistered live A/B with 24 locally graded sessions: the treatment arm reduces pooled cumulative input by 41.5% (14.87M to 8.70M tokens; four-task cluster-bootstrap 95% interval 31.1–54.8%) and passes a frozen operational quality rule with 9/12 versus 10/12 successes, but an admission-only counterfactual predicts at least this reduction, and unaligned history edits increase cache writes 68.1%, leaving only 2.5% lower CLI cost. B7 fits a message-extent cache model that is exact on 11/12 append-only native sessions. A replay rule that differs from the shipped scheduler models a 61.5% pooled dollar reduction on 36 well-calibrated sessions, all of it from 8 interactive sessions (three carry 76.5%); the median session gains nothing, and the result is not live. In B8 v2 (three pairs of chained three-task sessions), the treatment cut list-price cost 29.34% (client-reported cost 29.22%) with 9/9 successes per arm, on tasks chosen after B6 that exclude the one task where B6 treatment failed. A 29.5% model figure agrees to 0.16 points but was committed together with the results, so it is a post-hoc check, and pair-level reductions span 5.9–44.9%. The gateway’s only fire per session was a cold start with nothing pending, so the saving is admission—disallowing 25 built-in tools and three MCP servers—measured against stock Claude Code, not against its tool search. Workload, dollars, task quality, and mechanism attribution therefore require separate claims.

1 Introduction

An agent that calls a language model 50 times does not pay for an early tool schema or file read once. The object remains in the request prefix and is processed on later calls, often as a discounted cache read. This creates two optimization opportunities and one trap. Admission control can prevent a large fixed object from entering at all. Lifetime control can replace an obsolete object after it has served its purpose. But a lifetime edit changes the byte prefix, potentially converting cheap cached input into an expensive cache write. With the one-hour cache tier used by the evaluated client, reads cost $0.1\times$ base input and writes $2\times$ [5]. Deleting context at the wrong time can lower token residency and increase dollar cost.

Prior work attacks important pieces of this problem. LLMingua-family methods compress supplied prompts [14, 15, 27]; MemGPT and recent coding agents manage or compact memory [26, 21, 32, 41]; tool retrievers avoid exposing a large catalog [8, 11]; and serving systems reuse KV state [19, 42, 13]. Recent API-level studies directly measure prompt-cache strategy [23, 29, 18], and a gateway-level manager already prices lifecycle eviction against cache hits and misses [35]. ContextRuntime occupies the client-side systems boundary between these areas: it does not train a compressor or control provider memory. It decides what request content to admit, what prior content can be replaced, and when a replacement is worth its cache consequence.

The design emerged from a measured elimination process, not a chosen final architecture. Search output compression preserved evidence locally but was too small in the whole-session denominator. A semantic code-reading surface received zero voluntary use in 11 runs. Graph retrieval did not clear a joint recall–budget gate. An offline discovery call-collapse oracle looked promising, yet its forced eager implementation raised live pooled input 55.6%. Those failures moved the system toward two mechanisms with large, prospective surfaces: shrink the fixed prefix before turn one and control the lifetime of already admitted objects. The companion measurement paper details this evidence lineage.

This paper contributes:

1. A deployable gateway that separates a forward-only safety planner from environment-specific history mutation, with observation, enforcement, persistence, recovery handles, and fail-open behavior.
2. A cache-aligned scheduling rule that compares future read savings with the cost of rewriting the divergent

suffix, rather than mutating at a fixed token or turn threshold—a priced, dynamic generalization of static clearing thresholds. We also report that the shipped suffix estimate prevents its break-even branch from firing on the Anthropic cache profiles.

3. A message-extent cache simulator calibrated against live partial-prefix hits and an explicit price profile; its append-only branch matches 11 of 12 live native sessions exactly, its edit branch has no committed calibration, and it is used only for modeled claims.
4. A staged evaluation: 24 live, locally graded B6 sessions, a 66-session B7 replay, and a live B8 run whose 29.34% pooled cost reduction agrees with a post-hoc model figure in aggregate but ranges from 5.9% to 44.9% across pairs.
5. An honest decomposition: B6 demonstrates context-workload reduction whose lifetime share is unidentified; B8 demonstrates a live dollar reduction from admission, a client launch flag, while the gateway applied no mutation; the giant-session lifetime payoff (61.5% on a calibration-filtered subset, 49.7% over all 54 replayed sessions) remains modeled.

The artifacts and executable runtime are public [7]. The live results come from one repository (Django), one model (Sonnet), one client version (Claude Code 2.1.229), and one machine, with four tasks in B6 and three in B8, so we treat them as design-partner evidence requiring independent replication.

2 Related Work

2.1 Prompt compression and context management

Prompt compression removes low-information text or learns a compact representation. LLMingua reports up to $20\times$ compression; LongLLMingua is query-aware and targets position bias; LLMingua-2 trains a task-agnostic token classifier [14, 15, 27]. RECOMP compresses retrieved evidence and can suppress irrelevant augmentation, while ICAE and gist tokens learn latent summaries [36, 12, 25]. These methods are plausible operators inside a runtime, but a per-prompt ratio does not decide which agent objects to target or whether a query-dependent rewrite destroys a discounted prefix.

Agent-memory work supplies the semantic control plane. MemGPT uses virtual-memory tiers [26]. CAT makes context maintenance a callable tool and reports strong SWE-bench-Verified accuracy under a bounded workspace [21]; Focus gives the agent explicit checkpoint/consolidate operations and reports 22.7% fewer tokens on five instances at unchanged 3/5 success [32]; AutoCompact learns a compaction policy [41]. StateComp trains a router that decides when history can be compressed, reporting 52.27% fewer total tokens on WorkBuddyBench with task performance preserved; it does not model pricing or caching [33]. A harness-design study compares five context-management strategies and four window budgets in 176 matched settings on SWE-bench Verified and Terminal-Bench and finds rule-based elision followed by summarization the

most efficient; its costs use OpenRouter rates without prompt-cache pricing [10]. TRACE shows that recurrent compression increases execution instability and optimizes the compaction prompt with a verifier, without cost accounting [24]. VERA reports 31.5–63.1% fewer cumulative non-cache tokens and notes that this is not a monetary cost [31]—the token-dollar gap that B6 measures live. Rule-based elision, learned timing, and compaction reliability are therefore established. ContextRuntime’s distinguishing elements are narrower: the timing decision is priced against provider cache economics, and failure handling preserves the original request at a black-box client boundary. The approaches are complementary: learned summaries or a learned timing policy can become another mutation type governed by the same cache gate.

2.2 Tool admission and coding-agent interfaces

Large tool catalogs create a fixed-prefix tax. Re-Invoke, ToolRerank, and MCP-Zero retrieve a small candidate set instead of presenting every schema [8, 43, 11]. Tool Attention gates tools dynamically and loads schemas lazily, reporting 95% fewer per-turn tool tokens on a simulated 120-tool benchmark, with end-to-end gains projected rather than measured [28]. Provider tool search makes deferral first-party: definitions marked `defer_loading` stay out of the context and system-prompt prefix and load on demand, and the documentation reports a typical reduction above 85% for a multi-server setup of about 55k tokens, with the prefix untouched so caching is preserved [6, 1, 4]. Admission is therefore a crowded, first-party mechanism. Our admission filter is simpler—it disallows 25 built-in tools and three MCP servers known unused for the evaluated headless task family—and it is measured end to end, but no arm compares it with native deferral or tool search, which remains the open comparison. It also exposes a deployment interaction: Claude Code normally defers MCP tools, yet falls back to eager loading when `ANTHROPIC_BASE_URL` targets a non-first-party proxy unless tool search is explicitly configured [2].

Coding-agent research shows why task evaluation cannot be replaced by token metrics. SWE-bench uses real repository issues and objective tests [16]. SWE-agent demonstrates that the agent-computer interface changes success [37]; Agentless obtains strong results with a simpler localization–repair–validation pipeline [34]; RepoCoder, GraphCoder, and LocAgent improve repository context selection [40, 22, 9]. ContextRuntime holds the prompt and grading protocol constant between arms so that an efficiency intervention must retain issue-resolution quality; the treatment’s admission flag does change the tool surface, so that quality must survive a capability restriction.

2.3 KV caching and API economics

vLLM/PagedAttention, SGLang/RadixAttention, Prompt Cache, ChunkAttention, RAGCache, CacheBlend, Preble, and Parrot optimize memory placement, prefix sharing, prefill, or application-aware scheduling inside an inference service [19, 42, 13, 39, 17, 38, 30, 20]. A proprietary API client cannot move KV pages; it can observe usage fields

and preserve or change request bytes.

Concurrent work studies this boundary directly. Don’t Break the Cache compares cache strategies across three providers and more than 500 research-agent sessions [23]. CAPC combines query-agnostic compression with explicit cache control and a two-tier cost model [29]. Keepalive economics analyzes whether paying to refresh a prefix across idle gaps is rational [18]. ContextRuntime adds coding-agent object lineage, mutation/recovery safety, partial interior-hit calibration, and live measurements in which token and dollar effects diverge.

TokenPilot is the closest prior work: a gateway-level, cache-efficient context manager for LLM agents that keeps a stable prefix and evicts content by lifecycle, priced against cache hits and misses; its abstract reports cost reductions of 61% and 56% in an isolated mode and 61% and 87% in a continuous mode on PinchBench and Claw-Eval [35]. TokenPilot states no explicit break-even rule. Anthropic’s context editing (`clear_tool_uses_20250919`) replaces older tool results with placeholders once a token trigger is reached (100k by default) and keeps the most recent N (3 by default); its `clear_at_least` parameter sets a minimum clearing size because clearing invalidates cached prefixes [3]. Forward-only tool-result retirement is therefore also first-party. Against this work, ContextRuntime contributes a live token-versus-dollar measurement on an unmodified commercial coding client, admission by removal, and an explicit priced inequality for history mutation (Equation 1), a dynamic generalization of the static `clear_at_least` threshold—whose shipped form, we show, cannot yet fire on the Anthropic profiles.

3 System Design

3.1 Goals and non-goals

The gateway follows six requirements.

1. **Preserve task semantics.** Exact edit preconditions and the most recent object for a warm key remain; a retired payload is replaced by a recovery handle.
2. **Use only forward information.** Production decisions cannot use the future labels available to an offline oracle.
3. **Separate safety from timing.** The planner decides *what* is eligible; the cache scheduler decides *when* to expose a new byte divergence.
4. **Keep the mutated stream stable.** Once a mutation fires, it is reapplied to every subsequent request so history does not snap back.
5. **Fail open.** Parsing, logging, or upstream rejection must not block the agent; the unmodified body is the recovery path.
6. **Expose evidence grades.** Live, modeled, and counterfactual results remain distinct.

We do not train a semantic compressor, change the model, promise cross-provider cache equivalence, or claim that discarded content is universally irrelevant. The runtime is a policy and accounting layer.

3.2 Admission control

Tool schemas are unusually leveraged because they are fixed, early, and often unused. The evaluated admission profile launches the client with its `-disallowedTools` mechanism for 25 built-in tools and three MCP servers (personal and company connectors) that never appeared in the headless task traces; the gateway itself performs no admission. A zero-model-call capture through the proxy reduced the transmitted tool set from 82 schemas (46.1k cl100k-estimated tokens) to six (5.6k), but that baseline was captured with the client’s native MCP-schema deferral switched off by the custom base URL (Section 7.1). Against the stock client, which defers MCP schemas, the measured first-request saving is 23,424 tokens: 41,554 tokens for a native first request (B8 v1) versus 18,130 with admission (B6). The filter is configured outside the model and therefore does not require voluntary tool-search adoption.

Admission is conservative. The six retained tools provide repository reading, search, editing, and execution needed by the task harness. The system never silently removes a schema selected by the active profile. The removed set includes general capabilities such as subagents and web search, so admission is also a capability restriction, and the size of its saving depends on how many unused tools and servers an environment exposes. Other deployments should derive their own allowlist from observe-mode traces; the particular 82-to-6 profile is not a universal recommendation.

3.3 Forward retirement planner

The planner observes tool-result objects in call order. Each object has an identifier, admission turn, estimated token size, a supersession key such as `path:module.py` or a command signature, and an optional recovery reference. A later object with the same key makes earlier objects mechanically *superseded*. The most recent object for a key is retained while the key is warm. If a key has not been touched for $L = 5$ turns, its older object becomes a *cold-tail* candidate. Plans are normally batched at $K = 10$ turns to amortize rewrites.

The mutator replaces an eligible `tool_result` with a small explicit stub, not an invented summary. The stub names a content-addressed handle or instructs the agent to rerun/re-read the source. The planner is pure and environment-independent. A subscription client that cannot rewrite history uses an unsupported mutator; a gateway or custom loop that owns the message array uses the in-process mutator. This seam prevents an offline policy from being confused with a deployable mechanism.

3.4 Thinking lifetime

Models can retain prior thinking or redacted-thinking blocks in later requests. A keep- N rule removes those blocks from all but the most recent N assistant messages; the experiments use $N = 1$. Thinking removal is tracked separately because displayed transcript text can omit the payload while provider usage still reflects it. Under cache

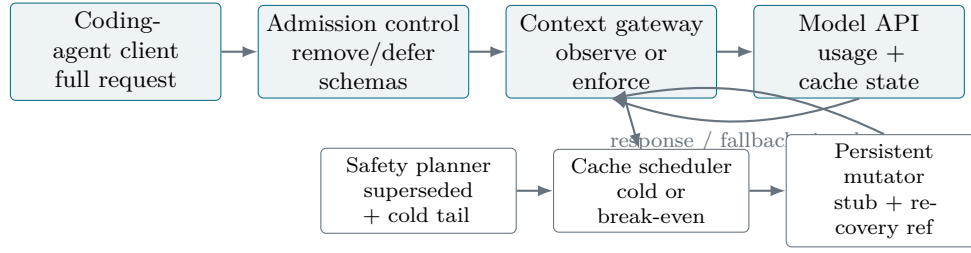


Figure 1: ContextRuntime request path. Admission changes the first request and remains prefix-stable; in the evaluated configuration it is a client launch flag (`-disallowedTools`), and the gateway itself performs no admission. The gateway reconstructs objects, asks the forward planner what is eligible, asks the scheduler whether a new cache divergence pays, and applies the fired set persistently. Observation mode logs the same decision without mutation; all exception paths preserve the original request.

alignment, a thinking frontier advances only when a mutation fires, making the edited byte stream stable between fires.

3.5 Cache-aligned scheduler

Let G be the tokens in new pending retirements, S the estimated suffix tokens that would need to be recreated after the earliest edit point, E the expected remaining calls, r the cache-read price, and w the cache-write price. A mutation clears the break-even gate when

$$rGE \geq (w - r)S. \quad (1)$$

The default evaluated profile uses $r = 0.1$, $w = 2.0$, and $E = 8$. If each retired token requires one suffix token to be rewritten, a write requires 19 future reads to break even. In principle Equation 1 can still fire with $E < 19$ when one early divergence retires substantially more pending content than the suffix it rewrites. The shipped gateway, however, counts S from the message holding the earliest pending tool result, so the suffix contains the pending content itself and $S \geq G$. With $E = 8$ the gate then requires $0.8G \geq 1.9S$ on the one-hour profile ($0.8G \geq 1.15S$ on the five-minute profile), which cannot hold: on both Anthropic profiles the break-even branch cannot fire, and only cold-start and TTL-gap fires are possible. The B7 replay’s gated policy uses a different rule—it adds pending thinking tokens to the gain and measures the suffix from observed prefix deltas—so B7’s modeled gated savings describe that replay rule, not the shipped scheduler.

The scheduler has three modes: **off**, **cold**, and **gated**; the default is off. Cold starts are safe moments to establish the mutated stream. A TTL-gap rule can also fire when the cache is believed expired, although B8 v1 reveals that nominal TTL is soft at 65 minutes; deployments should prefer observed miss telemetry or a conservative margin. Gated mode adds Equation 1, subject to the limitation above. The scheduler decides timing only and cannot promote an unsafe object.

The key state invariant is persistence. A fired identifier enters a set and is rewritten on every later request. Reapplication is byte-stable and does not create another divergence. Only newly eligible objects can cause a new cache invalidation. This is designed to repair the B6 “flash” behavior, in which retirement applied only on batch-boundary requests and history returned on intervening

calls; the clean live run (B8 v2) applied no mutation, so the repair is not yet demonstrated live.

3.6 Operational safety

Modes form an escalation path: off passes through, observe logs opportunities, and enforce mutates. Unexpected parsing errors return the untouched request. The live harness also retries an original body if an upstream response rejects a mutated request. Logs record the eligible count, tokens, boundary, fired reason, gap, pending mass, suffix estimate, persistent reapplications, applied retirements, stripped thinking, and fallback count. These fields make “no harm” measurable, but only partly: the proxy logs a fallback only for a 4xx response to a mutated body, and 5xx responses and dropped connections leave no recorded outcome.

4 Evaluation Method

4.1 Research questions

RQ1:

Does integrated admission and lifetime control reduce cumulative live input while preserving objectively graded coding quality?

RQ2:

Can a calibrated cache model identify when lifetime mutation earns dollars and when it should hold?

RQ3:

Does a live run of the admission-plus-aligned stack agree with the calibrated model, and which component produces its saving?

4.2 B6: integrated live A/B

B6 contains four modern Django SWE-bench-Verified instances, three interleaved paired repetitions, and two arms, for 24 live sessions. Every run uses the same Sonnet model, prompt, budget cap, and fresh worktree. The native arm is stock headless Claude Code. The treatment arm launches the client with the admission flag of Section 3.2 and routes it through the gateway in enforce mode with lag-five/batch-ten retirement and thinking keep-one; there is no admission-only arm. Graph, discovery, and search-replacement mechanisms are excluded because earlier gates closed them. The protocol commit (2026-08-27) precedes the results commit (2026-08-28), and the protocol’s post-run diff changes only its status header.

The preregistered primary endpoint is

$$R = \frac{\sum_t P_{t,T}}{\sum_t P_{t,N}}, \quad P_t = R_t + W_t + U_t, \quad (2)$$

reported per task and pooled. API calls are reconstructed by merging transcript records with the same `requestId`; sidechains are excluded. A run succeeds only when the task’s FAIL_TO_PASS tests pass and its PASS_TO_PASS tests remain green, as judged by a local macOS test runner rather than the official SWE-bench harness. The runner skips 6 of the 25 PASS_TO_PASS tests as unresolvable, was repaired mid-run (agent edits to test files are reset before grading), and the grades for runs 16485-N0 and 16485-T0 were repaired post hoc by replaying the transcripts’ Edit/Write operations onto a fresh worktree. The frozen operational quality rule requires total treatment successes to be at least native successes minus one, with no task at 0/3 treatment when native is 3/3. This is a program decision rule, not a statistically powered non-inferiority trial.

No session times out or reaches the budget cap. We add a post-hoc task-cluster bootstrap interval for effect-scale uncertainty; four tasks make it intentionally coarse, and the preregistered pooled ratio remains the decision endpoint.

4.3 B7: calibrated replay

B7 simulates cached prefix extents with TTLs and partial interior hits. On append-only call t , the largest alive matching extent is read and the remaining suffix is created. When history first differs at depth c , the request may read the largest matching prefix before c ; stale longer extents are discarded and the new full extent is stored. Inputs are observed per-call API token totals, so text token estimation is not on the pricing critical path.

Calibration uses the 12 B6 native timelines and the 12 observed B6 mutation schedules. A second replay covers 54 real sessions—26 interactive sessions and 28 short headless Django runs from this project’s own experiments (at most 75 calls each)—with up to 2,315 calls and 1.2 billion cumulative input tokens per session. Because sessions with native compaction violate the simple append-only model, results are reported for all 54 and for a frozen well-calibrated subset of 36 with absolute cache-creation error at most 20%; that subset holds all 28 headless runs but only 8 of the 26 interactive sessions. Policies are native, persistent-but-unaligned, cold-only, gated, and an oracle using the true remaining-call count. The replay’s gated rule is not the shipped scheduler’s (Section 3.5).

We price calls in base-input-token equivalents (BITE):

$$\text{BITE} = 0.1R + 2.0W + 1.0U + 5.0O. \quad (3)$$

Percentage deltas are more portable than absolute dollars because the interactive archive mixes models.

4.4 B8: live validation and a post-hoc model check

B8 v2 contains three live pairs. Each session chains three graded Django tasks in one conversation, with a 60-second

pause between tasks. The native arm is unchanged. The treatment combines admission with the persistent gated gateway. The three tasks (16485, 16527, 16901) were fixed after B6’s results and exclude 16502, the only B6 task on which treatment failed (0/3 versus 2/3 native). The B7 machinery, applied to B6 native timelines with the measured 23,424-token per-call admission shift, predicts a 29.5% pooled BITE reduction with a validation band of 22–36% and a 46.3% residency reduction. The protocol section is dated before the run, but the prediction first appears in version control in the same commit as the v2 results (af822b7, 2026-09-01 12:47 IST); the v2 treatment sessions ran from 11:33 to 12:41 IST that day according to the gateway logs, and the committed v1 protocol (623b612, 2026-08-31) has no v2 section. We therefore report B8 v2 as a post-hoc check of the calibrated model, not as a preregistered test. The primary endpoint is pooled BITE; secondary endpoints include CLI-reported cost, cumulative input, fires by reason, fallbacks, and nine task grades per arm.

An earlier v1 experiment is preserved as a confounded negative result. Its treatment used a custom base URL without admission and 65-minute gaps. The proxy caused the client to eagerly include tool schemas, and the nominal one-hour TTL remained warm. Post-hoc correction does not rescue v1; the findings instead determine the v2 design.

5 B6: Live Workload Reduction

Table 1 shows the primary result. Pooled cumulative input falls from 14,866,113 to 8,699,786 tokens, $R = 0.5852$, a 41.5% reduction. All four task-level pooled effects are positive, ranging from 28.1% to 58.7%. Ten of 12 individual pairs reduce input; stochastic trajectory length produces two reversals and a wide pair range. The post-hoc task-cluster bootstrap 95% interval is 31.1–54.8%. With only four clusters, this interval is a sensitivity summary rather than a basis for population generalization.

The reduction cannot be attributed to the lifetime mechanisms. B6 has no admission-only arm, and an admission-only counterfactual—subtracting the 23,424-token per-call admission shift from each of the 279 native calls—predicts $R = 0.560$, a reduction at least as large as the observed $R = 0.585$. The live contribution of retirement and thinking removal is therefore unidentified; it could be zero or negative.

Objective quality is 10/12 native and 9/12 treatment, passing the frozen rule exactly at its bound. Task 16502 is the residual risk: treatment is 0/3 while native is 2/3. All four failures across arms share the same shallow-fix mode; one native and one treatment patch are essentially the same (agent patches were not frozen, so this comparison cannot be re-checked from the artifacts). The sample cannot rule out a treatment effect, and we do not use the cross-arm similarity to erase the failure. Task 16901 moves in the other direction, with treatment 3/3 and native 2/3.

The gateway logs record 353 retirement applications and 1,794 thinking-block strips, but these count cumulative re-applications; the distinct counts are about 193

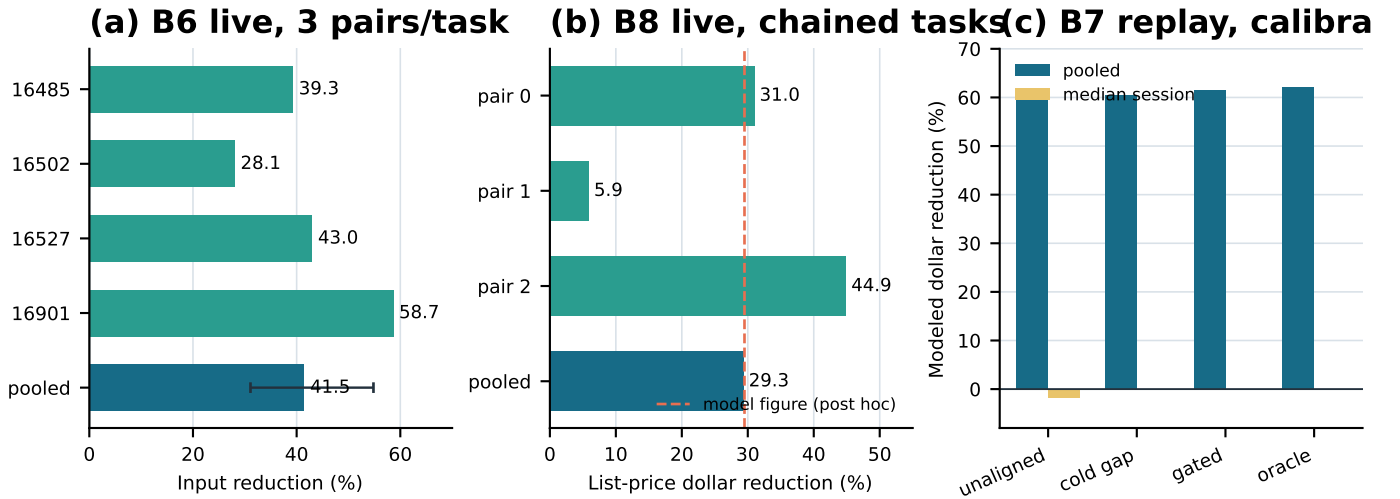


Figure 2: Three evidence stages. (a) B6 live cumulative-input reduction by task and pooled; the pooled whisker is a post-hoc four-task cluster-bootstrap 95% interval. (b) B8 v2 live BITE reduction by pair and pooled; the dashed line is the model’s 29.5% figure, which was committed together with the results and is therefore a post-hoc check. (c) B7 modeled dollar reduction on the 36 well-calibrated replay sessions: 28 short headless Django runs, which the gated policy leaves unchanged, and 8 interactive sessions that carry all of the gated saving. “Gated” is the replay’s rule, not the shipped scheduler. The pooled/median split shows that the modeled lifetime value is tail-concentrated. B7 values are not live savings.

Table 1: B6 preregistered primary endpoint and objective success counts. Input is the sum of cache read, cache creation, and uncached input over three runs per arm.

Task	Native input	Treatment input	Reduction	Success N/T
16485	2,772,225	1,682,341	39.3%	3/3
16502	4,753,322	3,418,567	28.1%	2/0
16527	3,606,230	2,055,051	43.0%	3/3
16901	3,734,336	1,543,827	58.7%	2/3
Pooled	14,866,113	8,699,786	41.5%	10/9

retired objects and at most 112 thinking blocks. No mutated request logged a fallback, but the proxy records a fallback only for a 4xx response to a mutated body, so 5xx responses and dropped connections are invisible; in session 16901-T2, 40 of 71 requests have no recorded outcome. The preregistered secondary endpoint of re-reads caused by retirement (reads of a path after the gateway retired that path’s object) was never computed. The only re-read figure, unconditional same-path repeat reads of 11/36 native versus 12/39 treatment, is not that endpoint and is recorded only in prose, so it cannot show whether retirement induces re-reads. The preregistered compaction-event endpoint was not reported. All runs are graded by executing FAIL_TO_PASS and PASS_TO_PASS tests rather than by process-success labels, with the local-runner caveats of Section 4.2.

RQ1 (live): The treatment arm (admission plus gateway) reduces cumulative input by 41.5% and passes the preregistered operational quality gate, but admission alone is predicted to account for at least this reduction. The result is a context-workload claim in four Django tasks, not a universal token or quality estimate, and not evidence for the lifetime mechanisms.

5.1 The dollar contradiction

B6 also exposes why cumulative input and dollars must be separate. Treatment makes 311 API calls versus 279 (+11.5%), lowers cache reads from 14,478,012 to 8,047,610 (44.4%), increases cache creation from 387,545 to 651,554 (68.1%), and increases output from 85,274 to 93,677 (9.9%). CLI cost moves only from \$7.98 to \$7.78, a 2.5% reduction. Repricing at list prices with one-hour cache writes reconstructs a 2.75% reduction, close to the client’s own cost report, which prices the same usage counts and is therefore not independent evidence.

The cause is observable in the gateway schedule. Retirement fires only on batch boundaries; the full original history reappears between them. Each boundary pays another suffix creation and the retirement saving lasts one call. Thinking removal and admission remain applied on every call and carry most of the residency effect. This “flash” mutation is the failure that B7 persistence and cache alignment are designed to repair.

6 B7: Cache Model and Scheduling

6.1 Calibration

Three platform facts are learned rather than assumed. First, the client requests the one-hour cache tier, so the

write multiplier is 2.0 rather than 1.25. Second, the provider serves partial interior hits: editing at depth x preserves the matching prefix before x . Third, back-to-back sessions can begin with a warm shared fixed prefix. Incorporating these facts makes the append-only simulator exact on 11/12 B6 native sessions; the remaining session, which contains a retry anomaly, is off by +103.7% in cache creation. This calibration is in-sample: the facts were learned on the same sessions. A replay of the 12 observed treatment mutation schedules was reported at about 1% cache-read error and 7.3% median absolute cache-creation error, but no committed artifact holds that calibration, so the edit branch, like any break-even fire, remains unvalidated. On the 54 replay sessions, median absolute creation error is 2.8% but p90 is 64%, revealing a compaction-heavy regime outside the model; the model fails the 20% calibration filter on 18 of the 26 interactive sessions.

6.2 Two workload regimes

On the 11 well-calibrated B6 native timelines, persistent unaligned mutation is modeled to increase dollars 8.4% while reducing residency 9.6%. Cold-only, gated, and oracle policies fire zero times and change cost by zero. Even an oracle finds no profitable mid-session edit in these short, hot sessions. Holding is the correct action.

On the 36 well-calibrated replay sessions—28 short headless Django runs and 8 interactive sessions—the picture changes:

Table 2: B7 cache-policy replay on the well-calibrated subset (28 headless Django runs, 8 interactive sessions). All values are modeled; “gated” is the replay’s rule, not the shipped scheduler.

Policy	Pooled $\Delta\$$	Median $\Delta\$$	$\Delta\text{res.}$	Fires
Unaligned	−61.9%	+1.4%	−74.2%	807
Cold gap	−60.4%	0.0%	−65.4%	125
Gated	−61.5%	0.0%	−67.0%	2,514
Oracle	−62.1%	0.0%	−73.7%	4,287

The pooled result is dominated by giant sessions. For gated scheduling, only 8/36 sessions have a positive modeled saving, and they are exactly the 8 interactive sessions; the 28 unchanged sessions are the headless Django runs, each at 0.0% modeled change. The top three sessions contribute 76.5% and the top five 90.4% of total modeled savings. The median session gains nothing. Across all 54 sessions, gated scheduling models a 49.7% pooled reduction, again with a median of zero. Unaligned scheduling reduces pooled cost yet regresses the median, exactly the behavior seen on B6-style timelines. Gated scheduling comes within 0.6 percentage points of the oracle’s pooled cost while avoiding negative modeled deltas, but it does not dominate on pooled dollars: unaligned scheduling models a slightly larger pooled reduction on the 36 sessions (61.9% versus 61.5%) and on all 54 (51.5% versus 49.7%). Gated scheduling’s advantage is distributional.

RQ2 (modeled): In replay, one rule adapts across regimes: hold in short cache-hot sessions, fire in the long tail when the read annuity dominates the rewrite. The 61.5% pooled payoff comes from 8 interactive sessions, describes the replay’s gated rule rather than the shipped scheduler, and remains a replay result, not a live claim.

7 B8: Live Admission Result and Post-Hoc Model Check

Pooled BITE falls from 2,183,665 to 1,542,873, a 29.34% reduction. The model’s figure is 29.5%, an error of −0.16 percentage points inside its 22–36% band, but because that figure was committed with the results (Section 4.4), this is a post-hoc agreement, not a preregistered validation. Nor does it show model precision: pair-level dollar reductions are 31.0%, 5.9%, and 44.9%, so a 0.16-point pooled agreement is well inside the replication variance that reporting all three pairs makes visible. The model’s residency figure missed: cumulative input falls 40.10%, from 14,158,182 to 8,480,551, against a modeled 46.3%. The client’s own cost report falls from \$6.553 to \$4.638, or 29.22%; it prices the same usage tokens with the same multipliers, so its agreement is not independent. Quality is 9/9 in both arms, but only on the three tasks where B6 treatment had already succeeded 3/3; B8 does not test non-inferiority on 16502, where B6 treatment failed.

Each treatment session’s only scheduler fire was a cold start at its first request, with nothing pending; no tool result was retired, no persistent stub was applied, and no thinking block was stripped. The absence of fallbacks is therefore vacuous here. The predicted break-even fires could not occur: as shipped, the break-even branch cannot fire on this profile (Section 3.5), so the holding was structural rather than a priced decision. The live cost saving comes from admission—the `-disallowedTools` launch configuration—and the proxy performs no admission. The model’s aggregate agreement reflects its admission arithmetic, a 23,424-token per-call shift applied to B6 native timelines; its fire-count and residency forecasts both missed.

RQ3 (live, post-hoc check): Launching the client with 25 built-in tools and three MCP servers disallowed, behind a gateway that applied no mutation, cut list-price cost 29.34%; the client’s own cost report shows 29.22%. A 29.5% model figure agrees in aggregate but was not preregistered, and pair-level reductions span 5.9–44.9%. The run supports admission pricing against stock Claude Code; it does not test cache-aware holding, admission against native tool search, or the B7 giant-session firing payoff.

7.1 The confounded v1 is part of the result

B8 v1 used a proxy without admission and 65-minute gaps. Its treatment moved far outside the frozen band, increasing list-price cost by 139.9% and 35.5% in its two pairs (76.5% pooled), because setting a custom base URL disabled normal MCP tool-schema deferral. Under v1’s preregistered rule, a cost increase refutes the gated stack; we reclassified the run as confounded after it finished, and

Table 3: B8 v2 primary and secondary live effects. Each pair contains three chained tasks per arm; all 18 task instances pass the same local grading as B6.

Pair	Native BITE	Treatment BITE	Dollar reduction	Residency reduction
0	894,926	617,611	31.0%	43.5%
1	551,222	518,781	5.9%	11.0%
2	737,518	406,481	44.9%	55.9%
Pooled	2,183,665	1,542,873	29.3%	40.1%

its preregistered lifetime-model prediction was never tested live. The first treatment request carried about 84,676 tokens versus 41,554 for native, followed by full-prefix rewrites as the tool list changed. One rewrite occurred when the gateway made no mutation, isolating the client behavior from the scheduler. Provider documentation now describes this fallback explicitly [2].

Native requests also retained cache hits after 65 minutes. Nominal TTL therefore did not identify a free boundary. Per the protocol, a post-hoc subtraction that would place one pair near the predicted band is not counted as validation. Instead, v1 yields two product rules: a proxy must preserve tool deferral or perform admission itself, and a time threshold alone is not proof that a cache is cold.

8 Discussion

8.1 The result is three claims, not one

The experiments support three different statements.

1. **Live workload:** admission plus unaligned lifetime control reduces cumulative input 41.5% in B6, with a frozen operational quality rule passed; admission alone is predicted to account for at least this reduction.
2. **Live dollars:** admission behind a gateway that applied no mutation reduces BITE 29.34% and client-reported cost 29.22% in B8; the realized saving is admission, measured against stock Claude Code rather than its tool search.
3. **Modeled lifetime dollars:** the replay’s gated rule reduces pooled cost about 61.5% in a calibrated replay whose saving comes from 8 interactive sessions, but the median is zero, the shipped scheduler does not execute that rule’s break-even branch, and no live firing-regime A/B exists.

Combining the largest percentages would be invalid. They refer to different workloads, policies, and evidence grades. The honest current product story is simpler: audit and shrink the fixed prefix first; observe cache behavior; enable persistent lifetime mutation only when Equation 1 clears, which the shipped estimate first needs to make possible; retain the original request as a safety path.

8.2 Why inaction is an optimization

Efficiency systems are often evaluated by how often they transform input. ContextRuntime’s key behavior is sometimes to do nothing. On a hot prefix, each deleted token saves r on a future read but can force $w - r$ of new creation in the divergent suffix. With $w - r = 1.9$ and a conservative remaining-call estimate, most ordinary ses-

sions cannot amortize the edit. A fixed threshold would mutate precisely when the cache makes mutation least attractive. In replay, holding removes B6’s modeled +8.4% cost regression. Live, B8 cannot credit the scheduler: its break-even branch could not fire, so holding was structural rather than a demonstrated decision, and without an admission-only arm (the native client with the same `-disallowedTools` flag and no proxy) the gateway’s own overhead or benefit in B8 is unmeasured.

8.3 Provider portability

The runtime parameterizes TTL, read, write, and output multipliers. Under free or cheap cache writes, Equation 1 fires much more often; under expensive long-TTL writes it holds. With the shipped suffix estimate ($S \geq G$) and $E = 8$, a break-even fire requires $rE \geq w - r$, which holds only for the unvalidated offline presets without a write premium, not for either Anthropic profile. The same session artifacts can be repriced, but provider profiles other than Anthropic one-hour are currently offline presets, not live validations. Cache-key construction, minimum cacheable length, eviction, and partial-hit semantics may differ, so portability requires a small calibration suite before enforcement.

8.4 Combining with learned compression

The planner currently replaces exact old payloads with recovery stubs. A learned compressor could reduce G earlier or preserve richer semantic state, and a learned timing policy such as StateComp [33] could propose when to compress. The scheduler remains useful: any generated summary changes bytes and should be introduced when its expected read annuity clears the rewrite cost. Conversely, cache-aware prompt compression such as CAPC [29] could improve admission while the retirement planner supplies object lifetime. The architectural boundary supports these compositions without treating a learned model as a safety proof.

9 Limitations and Future Work

Scale and scope. B6 has four task clusters and B8 three session pairs, all from one repository (Django), one model (Sonnet), one client version (Claude Code 2.1.229), and one machine. Large within-task variance is visible: B8 pair-level reductions span 5.9–44.9%. The operational quality gate is not a powered statistical non-inferiority test, grading uses a local runner rather than the official harness, and B8’s tasks exclude the one B6 task where treatment failed. Replication should expand repositories, languages,

models, and agents while preserving paired prompts and objective grading.

Attribution and baselines. No arm isolates admission from the gateway (the native client with `-disallowedTools` and no proxy), and no arm compares admission with the client’s own tool search; the native arm is stock Claude Code with its default MCP deferral. With a custom `ANTHROPIC_BASE_URL` that deferral is off (first request 84,676 versus 41,554 tokens), so the 82-to-6 schema baseline was measured against a de-deferred prefix. The size of the admission saving depends on how many unused tools and servers the environment exposes; ours included personal and company MCP connectors.

The tail is not live. B7’s 61.5% pooled payoff (49.7% over all 54 sessions) appears in 8 interactive sessions, three of which carry 76.5% of it, and is zero for the median. It is also computed with the replay’s gated rule, which differs from the shipped scheduler. A decisive next experiment must first make the shipped break-even branch able to fire, then enter that firing regime under controlled cost, preregister both pooled and distributional endpoints before any spend, and inspect recovery behavior.

Cache observability. The scheduler infers state from timing and a calibrated extent model. B8 v1 shows that nominal TTL is soft. Provider-supplied cache identifiers or explicit hit telemetry would make cold-boundary decisions more reliable. Until then, TTL-gap firing should use conservative margins or be disabled.

Token estimation off the critical path. Observed API usage prices sessions exactly, but pending retirement size and suffix depth use calibrated character/token estimators. Estimation error can change a knife-edge fire. B8’s predicted nine marginal fires versus zero observed is not such a case: because the shipped suffix estimate includes the pending content itself, the break-even branch could not fire. A safety margin or online calibration should accompany provider transfer.

Mutation semantics. Supersession is mechanically strong; cold-tail retirement is a policy assumption with a recovery path. The mutation-safety evidence is thinner than a count of 17 live enforce sessions suggests: only the 12 B6 sessions are auditable, all with cache alignment off; the 3 B8 v2 sessions applied no mutation; and the 2 B8 v1 sessions retain aggregates only. The proxy logs a fallback only for a 4xx on a mutated body, and in B6 session 16901-T2, 40 of 71 requests have no recorded outcome. The preregistered retirement-caused re-read endpoint was never computed. None of this proves semantic safety. Future work should log recovery-handle use, causal rereads, and all upstream failures over larger diverse workloads.

Private traces. The 54-session B7 archive is not redistributed, and raw transcripts for the B6–B8 sessions are not in the repository, so the B7 replay and the B8 prediction cannot be re-run from committed data. Aggregate replay output and source code are public, but independent replication requires new traces. The live B6/B8 aggregate artifacts and gateway decision logs are committed.

10 Conclusion

Context optimization in coding agents is not a single compression problem. Fixed schemas, old tool results, hidden reasoning, future turns, and provider cache state interact. ContextRuntime separates admission, safety, mutation, and timing so that each can be measured independently. Live B6 results show a 41.5% cumulative-input reduction, which admission alone is predicted to explain, and reveal that unaligned edits turn cache reads into writes and nearly erase dollar savings. The calibrated B7 model turns this contradiction into a break-even rule, but the shipped scheduler’s suffix estimate keeps that rule from firing on the Anthropic profiles. Live B8 then measures a 29.34% cost reduction that agrees in aggregate with a post-hoc 29.5% model figure; the gateway applied no mutation, so the saving is admission, a client launch configuration whose size depends on the unused tools an environment exposes and which has not been compared with the client’s own tool search. Today, admission is the demonstrated dollar lever; cache-aligned lifetime savings in giant sessions remain a testable hypothesis rather than a marketed result.

Artifact and Reproducibility Note

Code and artifacts: <https://github.com/ankit961/LLM-Token-efficiency>

The repository contains the B6 and B8 protocols, task-level live artifacts, treatment decision logs, B7 cache simulator, provider profiles, scheduler and gateway code, regression tests, and a script that recomputes all tables and figures without model calls; raw session transcripts are not included. From the repository root, run `python3 papers/scripts/generate_assets.py`, then build this source with `latexmk -pdf main.tex`.

References

- [1] Anthropic. Introducing advanced tool use on the claude developer platform. Engineering blog, 2025.
- [2] Anthropic. Connect claude code to tools via MCP. Claude Code documentation, 2026.
- [3] Anthropic. Context editing. <https://platform.claude.com/docs/en/build-with-claude/context-editing>, 2026. Claude Platform documentation, accessed 2026-10-05.
- [4] Anthropic. Manage tool context. Claude Platform documentation, 2026.
- [5] Anthropic. Prompt caching. Claude Platform documentation, 2026.
- [6] Anthropic. Tool search tool. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool>, 2026. Claude Platform documentation, accessed 2026-10-05.
- [7] Ankit Chauhan. LLM Token Efficiency (ContextRuntime). GitHub repository, 2026. Evidence snapshot at commit 4656e8c.

- [8] Yanfei Chen, Jinsung Yoon, Devendra Singh Sachan, Qingze Wang, Vincent Cohen-Addad, Mohammadhossein Bateni, Chen-Yu Lee, and Tomas Pfister. Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In *Findings of EMNLP*, pages 4705–4726, 2024.
- [9] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of ACL*, pages 8697–8727, 2025.
- [10] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents, 2026.
- [11] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero: Proactive toolchain construction for LLM agents from scratch, 2025.
- [12] Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations*, 2024.
- [13] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, 2024.
- [14] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, pages 13358–13376, 2023.
- [15] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of ACL*, pages 1658–1677, 2024.
- [16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- [17] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation, 2024.
- [18] Maxim Khailo. Keeping the cache warm pays: Keepalive economics for agentic workloads, 2026.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023.
- [20] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 929–945, 2024.
- [21] Shukai Liu, Bo Jiang, Jian Yang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon SWE-agents. In *Findings of ACL*, pages 20604–20617, 2026.
- [22] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. GraphCoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, pages 570–581, 2024.
- [23] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks, 2026.
- [24] Guanghui Min, Liang Wu, Mayank Darbari, Chen Chen, and Liangjie Hong. Toward reliable context compression for long-horizon agents: An empirical study of execution instability, 2026.
- [25] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, 2023.
- [26] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.
- [27] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*, pages 963–981, 2024.
- [28] Anuj Sadani and Deepak Kumar. Tool attention is all you need: Dynamic tool gating and lazy schema loading for eliminating the MCP/tools tax in scalable agentic workflows, 2026.
- [29] Yan Song. Cache-aware prompt compression: A two-tier cost model for LLM API caching, 2026.

- [30] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for LLM serving, 2024.
- [31] Jiaqi Su, Cong Pang, Jiawei Hong, Tiankuo Yao, Zixuan Chen, Xin Lou, and Lewei Lu. When history is multimodal: Rethinking context management for long-horizon agents, 2026.
- [32] Nikhil Verma. Active context compression: Autonomous memory management in LLM agents, 2026.
- [33] Mingxuan Wang, Hongyue Chen, Yinglong Guo, Fei Luo, Chao Ning, Bo Wang, Guorun Yao, Yanbiao Ma, and Jungong Han. StateComp: Learning when to compress history in long horizon agents, 2026.
- [34] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025.
- [35] Buqiang Xu, Zirui Xue, Dianmou Chen, Chenyang Fu, Chiyu Wu, Caiying Huang, Chen Jiang, Jizhan Fang, Xinle Deng, Yijun Chen, Yunzhi Yao, Xuehai Wang, Jin Shang, Gong Yu, and Ningyu Zhang. TokenPilot: Cache-efficient context management for LLM agents, 2026.
- [36] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *International Conference on Learning Representations*, 2024.
- [37] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [38] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of EuroSys*, 2025.
- [39] Lu Ye, Ze Tao, Yong Huang, and Yang Li. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In *Proceedings of ACL*, 2024.
- [40] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of EMNLP*, pages 2471–2484, 2023.
- [41] Xuan Zhang, Longtao Zheng, Cunxiao Du, Bo An, and Xin Dong. AutoCompact: Learning when to compact context in long-horizon coding agents. Project report, 2026.
- [42] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs, 2024.
- [43] Yuanhang Zheng, Peng Li, Wei Liu, Yang Liu, Jian Luan, and Bin Wang. ToolRerank: Adaptive and hierarchy-aware reranking for tool retrieval. In *Proceedings of LREC-COLING*, pages 16263–16273, 2024.